

1 Resumen del Proyecto

Este documento detalla el pipeline de entrenamiento de un modelo de Machine Learning (Random Forest) diseñado para predecir el tipo de plan nutricional recomendado para pacientes.

El script de Python asociado automatiza el proceso completo, desde la carga de datos y el preprocesamiento, hasta el entrenamiento, la evaluación, la visualización de resultados y el guardado del modelo final para su uso en producción.

2 Dependencias Principales

El script se basa en las siguientes bibliotecas de Python:

- **Pandas & NumPy:** Para la manipulación y procesamiento de datos.
- **Scikit-learn (sklearn):** Para el modelo (`RandomForestClassifier`), la división de datos (`train_test_split`), la codificación (`LabelEncoder`) y las métricas de evaluación.
- **Matplotlib & Seaborn:** Para la generación de visualizaciones.
- **Pickle:** Para serializar y guardar el modelo entrenado.
- **Google Colab Files:** Para la carga y descarga interactiva de archivos en el entorno de Google Colab.

3 Estructura del Pipeline

El proceso de entrenamiento se divide en varias funciones modulares:

3.1 Carga de Datos (`cargar_dataset`)

Esta función inicializa el sistema y solicita al usuario que cargue el archivo CSV del dataset. Implementa un sistema robusto que prueba múltiples codificaciones (ej. `utf-8`, `latin-1`) para asegurar que el archivo se lea correctamente, independientemente de su formato.

3.2 Preprocesamiento (`preprocesar_datos`)

Esta es una fase crítica que realiza las siguientes tareas:

1. **Selección de Variables:** Define explícitamente las columnas que se usarán como *features* (predictoras) y cuál es la variable *target* (objetivo).
2. **Limpieza Numérica:** Convierte columnas numéricas que podrían estar como texto (ej. usando `'` como decimal) al formato numérico correcto.
3. **Manejo de Nulos:** Rellena cualquier valor nulo (NaN) resultante en las columnas numéricas utilizando la mediana de la columna, una estrategia robusta ante valores atípicos.
4. **Codificación Categórica:** Utiliza `LabelEncoder` para convertir las variables categóricas (como 'Sexo') y la variable objetivo (los planes nutricionales) en valores numéricos que el modelo pueda entender.

3.3 División de Datos (dividir_datos)

Divide el dataset preprocesado en dos conjuntos:

- **Entrenamiento (80%):** Datos utilizados para enseñar al modelo.
- **Prueba (20%):** Datos que el modelo no ha visto, usados para evaluar su rendimiento real.

Se utiliza `stratify=y` para asegurar que la proporción de cada plan nutricional sea la misma en ambos conjuntos, lo cual es crucial en datasets desbalanceados.

3.4 Entrenamiento del Modelo (entrenar_modelo)

Configura y entrena el clasificador `RandomForestClassifier`. Los hiperparámetros clave están fijados para garantizar resultados reproducibles:

```
modelo = RandomForestClassifier(  
    n_estimators=100,          # 100 árboles de decisión  
    max_depth=20,             # Profundidad máxima  
    min_samples_split=5,      # Mínimo para dividir un nodo  
    min_samples_leaf=2,       # Mínimo en una hoja final  
    random_state=42,          # Para reproducibilidad  
    n_jobs=-1                  # Usar todos los núcleos de CPU  
)
```

3.5 Evaluación del Modelo (evaluar_modelo)

Mide el rendimiento del modelo entrenado usando varias métricas:

- **Accuracy (Precisión):** Muestra el porcentaje de aciertos tanto en los datos de entrenamiento (para detectar overfitting) como en los de prueba (rendimiento real).
- **Validación Cruzada (5-fold):** Proporciona una medida más robusta del rendimiento promedio del modelo.
- **Reporte de Clasificación:** Detalla las métricas de *Precision*, *Recall* y *F1-Score* para cada tipo de plan nutricional, permitiendo identificar si el modelo tiene dificultades con alguna clase específica.

3.6 Visualización (visualizar_resultados)

Genera un panel de gráficos guardado como `resultados_modelo.png`, que incluye:

- Matriz de Confusión.
- Importancia de Variables (Feature Importances).
- Distribución de Predicciones.
- Comparación Real vs. Predicho.

3.7 Guardado y Documentación

- `guardar_modelo`: Empaqueta el objeto del modelo, los `LabelEncoders` (esenciales para revertir las predicciones a texto) y los nombres de las columnas en un único archivo `.pkl` usando `pickle`.
- `generar_documentacion`: Imprime un resumen ejecutivo en la consola (que sirve de base para este documento).

4 Variables (Features) Utilizadas

El modelo fue entrenado con un conjunto específico de 7 características, excluyendo deliberadamente otras variables (como 'Porcentaje de grasa corporal').

4.1 Variables Numéricas (5)

```
columnas_features_numericas = [  
    'Edad',  
    'Peso actual (kg)',  
    'Talla (m)',  
    'IMC (Índice de Masa Corporal)',  
    'Circunferencia de cintura (cm)'  
]
```

4.2 Variables Categóricas (2)

```
columnas_features_categoricas = [  
    'Sexo',  
    'Nivel de actividad física'  
]
```

4.3 Variable Objetivo (Target)

```
target_col = 'Tipo de plan nutricional recomendado'
```

5 Ejecución Principal

La función `ejecutar_pipeline_completo` orquesta la ejecución de todas las funciones anteriores en la secuencia correcta. El bloque `if __name__ == "__main__":` asegura que este pipeline se inicie automáticamente cuando el script es ejecutado.